

Hamza Teaches — Exam Portal

Developer Technical Specification

Version: 1.0 **Stack:** React 18 (single-file) · Supabase (Postgres) · Cloudinary · Vercel

Audience: The developer building, wiring, or maintaining this portal. **Companion docs:**

`DESIGN_AND_WIREFRAMES.md` (visual spec), `HamzaTeaches-DeveloperBrief.docx` (account setup + deploy steps).

0. Read this first — current state vs. target state

There are two builds in play. Know which one you are touching.

Build	File	Persistence	Use
Demo	<code>hamza-portal-demo.jsx</code>	In-memory only (<code>MEM</code> , <code>LSMEM</code>)	Renders in any React sandbox. Seeded data on load. For showing the product.
Deployment package	<code>hamza-portal/App.jsx</code>	<code>window.storage</code> key-value (client-side)	Has Supabase client + Cloudinary upload wired, but reads/writes still go through <code>window.storage</code> .

The integration gap (your main job for true production): `hamza-portal/App.jsx` imports `@supabase/supabase-js`, initialises a client, and uploads photos to Cloudinary — but the actual data load and persistence still run through `sload/ssave` → `window.storage`. The Supabase tables, triggers, and RLS exist in `schema.sql` but are **not yet called** for CRUD.

To ship a real multi-device product you must:

1. Replace each `sload(K.X)` read with the matching `supabase.from('table').select()`.
2. Replace each `ssave(K.X, ...)` effect with `supabase.from('table').insert()/update()/delete()`.
3. Move MCQ grading off the client and onto the Postgres trigger (already in `schema.sql`).
4. Keep the Cloudinary upload path (already correct).

Everything else — UI, state shape, business rules — is built and is documented below so you can wire the data layer without reverse-engineering the app.

1. Architecture

Single-file React 18 application. No router. No Redux. State lives in the top-level `App` component and flows down through props. View switching is driven by a `view` string and a few “selected entity” state slots, not by URL.

```
App (all state)
├─ unauthenticated → StudentLogin | AdminLogin
├─ student authenticated → StudentDash → TakeTest → Submitted
│                         → Results
│                         → StudentAnalytics
└─ admin authenticated → AdminDash → CreateTest
                        → TestDetail → GradeSub
                        → ManageStudents → StudentAnalytics
                        → ManageCohorts
                        → ManageAnnouncements
                        → QuestionBank
                        → AdminAnalytics → StudentAnalytics
```

Why single-file: the owner is non-technical and deploys by handing one component to a sandbox or to Vercel. Keep it single-file unless you are taking over long-term maintenance, in which case split by the component groups in §6.

2. Data models

These are the exact runtime shapes the UI expects. The Postgres schema (§7) maps to these. Types written TypeScript-style for clarity; the code is plain JS.

```
type ID = string;           // uid(): `${Date.now()}_${base36(4)}`
type Millis = number;       // Date.now() epoch ms

interface Cohort {
  id: ID;
  name: string;
  color: string;             // hex, from COHORT_COLORS
  createdAt: Millis;
}
```

```

interface Student {
  id: ID;
  user_id?: ID;           // Supabase Auth UUID (production)
  username: string;       // unique, login handle (no email required)
  password?: string;      // legacy/demo plaintext – DO NOT ship to prod
  salt?: string;          // PBKDF2 per-user salt (production)
  passwordHash?: string;  // PBKDF2-SHA256 hash (production)
  cohortId: ID | null;
  createdAt: Millis;
}

type QType = "mcq" | "text" | "photo";

interface Question {
  id: ID;
  number: string;         // "Q1", "Q2" ... display label
  type: QType;
  text: string;           // may be empty IF fileData present
  marks: number;
  topic: string | null;   // used for mastery analytics
  options?: string[];     // length 4, MCQ only
  correctOption?: 0|1|2|3; // MCQ answer key – never expose to student client
  fileData?: string | null; // base64 (demo) / Cloudinary URL (prod)
  fileType?: "pdf" | "image" | null;
  fileName?: string | null;
}

type Status = "draft" | "active" | "closed";
type AnswerMode = "per-question" | "single";

interface Test {
  id: ID;
  code: string;           // 4-char student access code, see code4()
  title: string;
  subject: string;        // one of SUBJECTS
  duration: number;       // minutes
  questions: Question[];
  answerMode: AnswerMode; // photo upload: per question, or one at end
  cohortId: ID | null;    // null = visible to all cohorts
  status: Status;
  openAt: Millis | null;  // scheduled open; null = manual
  closeAt: Millis | null; // scheduled close; null = manual
  createdAt: Millis;
}

interface Answer {
  questionId: ID;

```

```

    type: QType;
    value: string;           // MCQ: option index as string; text: the answer
    isCorrect: boolean | null; // MCQ only; null for text/photo
    marksAwarded: number | null; // null until graded (text/photo)
    photoData: string | null; // base64 (demo) / Cloudinary URL (prod)
    feedback: string | null; // teacher's per-question note
  }

  interface Submission {
    id: ID;
    testId: ID;
    studentId: ID;
    studentName: string; // denormalised snapshot at submit time
    submittedAt: Millis;
    timeUp: boolean; // true if auto-submitted on timer/close
    released: boolean; // results visible to student only when true
    totalMarksAwarded: number | null; // null until fully graded
    summaryPhoto?: string | null; // "single" answerMode upload
    answers: Answer[];
  }

  interface BankQuestion {
    id: ID;
    subject: string;
    topic: string | null;
    type: QType;
    text: string;
    marks: number;
    options?: string[];
    correctOption?: 0|1|2|3; // mirrors Question; bank items copy into tests
    createdAt: Millis;
  }

  interface Announcement {
    id: ID;
    text: string;
    pinned: boolean; // pinned = always shown, not dismissible
    createdAt: Millis;
  }

```

3. Application state (top-level `App`)

```

// data collections
const [tests, setTests] = useState([]);

```

```

const [submissions, setSubmissions] = useState([]);
const [students, setStudents] = useState([]);
const [bank, setBank] = useState([]);
const [announcements, setAnnouncements] = useState([]);
const [cohorts, setCohorts] = useState([]);
const [ready, setReady] = useState(false); // gates render until

// session
const [student, setStudent] = useState(null); // logged-in student o
const [adminAuth, setAdminAuth] = useState(false);
const [showAdmin, setShowAdmin] = useState(false); // admin login screen

// navigation
const [view, setView] = useState("sLogin");
const [activeCohortId, setActiveCohortId] = useState(null); // admin cohort filte

// selected entities
const [takingTest, setTakingTest] = useState(null);
const [viewingResult, setViewingResult] = useState({ t:null, s:null });
const [analyticsStudent, setAnalyticsStudent] = useState(null);
const [selTest, setSelTest] = useState(null); // admin test detail
const [selSub, setSelSub] = useState(null); // admin grading

```

Persistence wiring (current): six `useEffect` hooks mirror each collection to storage on change, gated by `ready`:

```

useEffect(()=>{ if(ready) ssave(K.T, tests); }, [tests, ready]);
// ...same for submissions, students, bank, announcements, cohorts

```

When you move to Supabase, replace the body of each effect (or better, the individual mutating actions) with the corresponding table write. Do **not** bulk-overwrite a whole table on every keystroke — convert to per-action `insert / update / delete`.

4. Storage keys (client build)

```

const K = {
  T: "ht_tests_v3",
  S: "ht_subs_v3",
  ST: "ht_students_v1",
  QB: "ht_qbank_v1",
  AN: "ht_announce_v1",
  CO: "ht_cohorts_v1",
};

```

Per-session (not collections):

- `ht_timer_{testId}_{studentId}` — `{ startedAt }` so a refresh resumes the same countdown.
- `ht_draft_{testId}_{studentId}` — in-progress answers.
- `ht_dismissed_{studentId}` — array of dismissed announcement ids.
- `ht_admin_lock` — `{ fails, until }` brute-force lock state.

In the demo these live in `LSMEM` (an object), since artifact sandboxes block `localStorage`. In production they can be real `localStorage` (device-local UX state) — they do **not** need to go to Supabase.

5. Design tokens (authoritative — copy from code, not from memory)

```
const C = {
  base:    "#181b1e", // page background
  bg:      "#1e2226", // card base
  surface: "#252a2e", // inputs, chips
  hover:   "#2d3238",

  border:    "rgba(255,255,255,0.06)",
  borderHi:  "rgba(251,193,89,0.25)",

  accent:    "#FBC159", // brand gold
  accentDark: "#e8a830",
  accentDim:  "rgba(251,193,89,0.09)",

  white:     "#fff",
  text:      "#e8eaec", // primary text
  mid:       "#9ca3af", // secondary text
  muted:     "#6b7280", // tertiary / meta

  success:   "#6FBBA8", successDim: "rgba(111,187,168,0.1)",
  error:     "#E5685A", errorDim:   "rgba(229,104,90,0.1)",
  blue:      "#7AA7D9",

};

const F = {
  ui:      "'Geist','Inter',system-ui,sans-serif",
```

```

    mono: "'Geist Mono','Courier New',monospace",
    hand: "'Caveat',cursive",    // login slogan only
  };

const GR = {
  bg:    "radial-gradient(ellipse at 50% 0%, #2a2d32 0%, #181b1e 65%)",
  card:  "linear-gradient(145deg, #262b30 0%, #1e2226 100%)",
  stat:  "linear-gradient(145deg, #2a2f34 0%, #1e2226 100%)",
  accent: "linear-gradient(135deg, #FBC159 0%, #e8a830 100%)",
};

const SH = {
  card:  "0 4px 20px rgba(0,0,0,0.5), inset 0 1px 0 rgba(255,255,255,0.05)",
  lift:  "0 8px 32px rgba(0,0,0,0.6), 0 0 20px rgba(251,193,89,0.07), inset 0 1px",
  glass: "0 24px 60px rgba(0,0,0,0.7), 0 0 80px rgba(251,193,89,0.04)",
  stat:  "0 2px 12px rgba(0,0,0,0.4), inset 0 1px 0 rgba(255,255,255,0.05)",
  modal: "0 32px 80px rgba(0,0,0,0.8), 0 0 0 1px rgba(255,255,255,0.06)",
  glow:  (col) => `0 0 30px ${col}22`,
};

const COHORT_COLORS = ["#FBC159", "#6FBBA8", "#7AA7D9", "#E5685A", "#a78bfa", "#fb923c"]

```

Note: `DESIGN_AND_WIREFRAMES.md` lists rounded values (e.g. base `#1a1d20`, grades A–F). Where the two disagree, **the code values above win**. Card radii in code are 12–14px; grade bands are Cambridge-style (§6.2).

6. Component catalog

6.1 Primitives (shared)

Component	Props	Responsibility
Btn	children, onClick, v, sm, full, icon, disabled, style	Button. v: primary (default) ghost subtle success danger teal warn.
Badge	children, col, dot	Status pill. col: green yellow red blue dim.
Field	label, children, err, hint	Form field wrapper with uppercase label + error/hint.
	label, value, onChange,	

TI	<code>type, placeholder, err</code>	Text input bound to a string state.
StatCard	<code>label, value, sub, col, icon, mobile</code>	Dashboard metric tile (gold mono value).
Nav	<code>title, sub, right, mobile</code>	Sticky top bar with gold gradient hairline.
Wrap / Body	<code>children, max, mobile</code>	Page shell + centered content column.
Mono	<code>children, style</code>	Monospace span.
Hr / SectionLabel	<code>— / children, action</code>	Divider / uppercase section header.
Toggle	<code>on, onChange, label</code>	iOS-style switch.
FileDisplay	<code>fileData, fileType, fileName</code>	Renders attached PDF (<code><embed></code>) or image.
SchedulePill	<code>test</code>	"Opens in / Closes in" countdown; red if <1h to close.
CohortDot	<code>cohort</code>	Colour dot + cohort name.
Loading	<code>—</code>	Spinner shown until <code>ready</code> .

6.2 Student components

Component	Key props	Responsibility
StudentLogin	<code>students, onLogin, onAdmin</code>	Username + password. <code>onAdmin</code> = hidden dot to reveal admin login.
StudentDash	<code>student, tests, submissions, announcements, onTakeTest, onViewResult, onProgress, onLogout</code>	Announcements, in-progress resume banner, stat cards, grades, "Up next" (open tests), "Coming up" (scheduled), and full submission record.
TakeTest	<code>test, student, onSubmit</code>	Timed test runner. See §6.4.
Submitted	<code>student, onDash</code>	Post-submit confirmation.

Results	test, sub, onBack	Released result: score, grade, per-question correctness + feedback.
StudentAnalytics	student, tests, submissions, onBack, isAdmin	Score trend chart, topic mastery bars, weak-topic flags. Shared with admin (isAdmin toggles framing).

6.3 Admin components

Component	Key props	Responsibility
AdminLogin	onLogin, onBack	Password gate. Brute-force lock + session timeout in prod.
AdminDash	tests, submissions, students, cohorts, announcements, activeCohortId, setActiveCohortId, onNew, onSelect, onStudents, onBank, onCohorts, onAnnouncements, onExport, onInsights, onLogout	Cohort filter, stat cards (students/tests/needs-grading/average/awaiting-release), test list with codes + status.
CreateTest	onSave, onBack, bank, cohorts	Build test: title, subject, duration, cohort, answerMode, schedule, questions. Save as draft or activate.
QBuilder	q, errs, onUp, onUpOpt, onRemove	Single-question editor inside CreateTest.
BankPicker	bank, onAdd, onClose	Modal: pick bank questions into a test.
TestDetail	test, subs, students, cohorts, onBack, onGrade, onToggle, onDelete, onBulkRelease	Access code, status toggle, MCQ accuracy per question, question chips, submission list with grading state.
		Per-question marks +

GradeSub	test, sub, students, onBack, onSave, onRelease	feedback. MCQ auto-shown. Save progress or release.
ManageStudents	students, setStudents, cohorts, onBack, onViewAnalytics	Add/remove students, reset password, assign cohort, open analytics.
StudentRow	s, cohorts, onRemove, onReset, onCohort, onViewAnalytics, mobile	One roster row.
ManageCohorts	cohorts, setCohorts, students, tests, onBack	Create/edit/delete cohorts (name + colour).
ManageAnnouncements	announcements, setAnnouncements, onBack	Create/pin/delete announcements.
AnnouncementCard	a, onDismiss	One announcement; dismiss button only when not pinned.
QuestionBank	bank, setBank, onBack	CRUD reusable questions.
BankQForm	onSave, initial, onCancel	Add/edit a bank question.
AdminAnalytics	tests, submissions, students, cohorts, activeCohortId, onBack, onViewStudent	Class average, grade distribution, per-test averages, class topic weakness, student leaderboard (click → StudentAnalytics).

6.4 TakeTest runtime (the most complex component)

- **Timer:** initial seconds = duration*60 unless ht_timer_{t}_{s} has a startedAt , in which case it resumes from elapsed. Persists startedAt on first mount.
- **Auto-submit on close:** setInterval every 10s checks Date.now() > test.closeAt ; if past and not yet submitted, calls handleSubmit(true) .
- **Auto-submit on timer zero:** countdown reaching 0 calls handleSubmit(true) .
- **Submit shape:**

```

onSubmit({
    timeUp,                // boolean
    answers: final,        // Answer[]
    summaryPhoto: summaryPhotoRef.current,
    totalMarksAwarded: null, // graded later (MCQ filled by trigger/client)
    released: false,
});

```

- **Guard:** `submittedRef` prevents double submit.
- **answerMode:** `per-question` shows an upload per photo question; `single` shows one summary upload at the end.

7. Business logic & rules (exact)

7.1 Grade bands (`gradeOf(got, max)`)

```

pct = got/max*100
<=90 A*   >=80 A   >=70 B   >=60 C   >=50 D   else U

```

Grade colours (`gradeCol`): A*→accent, A→success, B→blue, C→mid, D→#f59e0b, U→error.

7.2 Topic mastery bands (`topicBand(pct)`), constants `TOPIC_WEAK=60` ,

`TOPIC_CRIT=50`

```

<50  crit    (red)
<60  weak    (#f59e0b)
<75  ok      (blue)
>=75 strong  (success)

```

7.3 MCQ auto-grading

```

isCorrect    = chosenIndex === question.correctOption
marksAwarded = isCorrect ? question.marks : 0

```

Demo does this client-side. **Production must do this in the Postgres trigger** so `correctOption` never reaches the student client.

7.4 Test open check (`isTestOpen(t)`)

```
status === "active"
  && (openAt == null || now >= openAt)
  && (closeAt == null || now <= closeAt)
```

7.5 Status transitions (TestDetail toggle)

```
active → closed
draft → active
closed → active
```

Toggle button label: active→"Close test", draft→"Activate", closed→"Reopen".

7.6 Submission grading state (`progress(sub)` , TestDetail)

```
non-mcq answers = answers where type !== "mcq"
none             → "graded"          (pure MCQ test)
none graded yet  → "pending"
some not all     → "partial"
all graded       → "graded"
```

7.7 "Needs grading" count (AdminDash)

```
submissions where !released AND some non-mcq answer has marksAwarded == null
```

7.8 Access code (`code4()`)

4 chars from `ACDEFGHJKLMNPQRTUVWXY34679` (ambiguous chars `B/I/O/S` and `0/1/2/5/8` excluded).

7.9 Validation (CreateTest)

- title required; duration ≥ 1 ; at least one question.
- each question: text required **unless** a file is attached.
- MCQ: at least 2 non-empty options.

7.10 Bulk release

Only offered when **every** question in the test is MCQ and there are unreleased submissions
→ one click releases all.

7.11 Constants

```
SUBJECTS = ["AS Accounting", "IGCSE Accounting", "AS Business",  
            "IGCSE Business", "AS Economics", "IGCSE Economics"];  
OL = ["A", "B", "C", "D"]; // MCQ option letters
```

8. Supabase data layer (target contract)

`schema.sql` defines 8 tables matching §2: `cohorts`, `students`, `tests`, `questions`, `submissions`, `answers`, `announcements`, `question_bank`.

8.1 Load (replace the six `sload` calls)

```
const [tests, subs, students, bank, announce, cohorts] = await Promise.all([  
  supabase.from('tests').select('*', questions('*')),  
  supabase.from('submissions').select('*', answers('*')),  
  supabase.from('students').select('*'),  
  supabase.from('question_bank').select('*'),  
  supabase.from('announcements').select('*'),  
  supabase.from('cohorts').select('*'),  
]);
```

8.2 Writes (replace the six mirror effects with per-action calls)

- Create test → insert `tests`, then insert its `questions` rows.
- Submit test → insert `submissions`, then insert `answers` (MCQ graded by trigger).
- Grade → update `answers.marks_awarded + answers.feedback`; `submissions.released`.
- Cohorts / announcements / students / bank → straightforward insert/update/delete.

8.3 Triggers (already in `schema.sql`)

- `grade_mcq_trigger` — BEFORE INSERT on `answers`: compares chosen index to `questions.answer_key_index`, sets `is_correct + marks_awarded`. Answer key stays server-side.
- `sum_marks_trigger` — AFTER INSERT/UPDATE on `answers`: recomputes `submissions.total_marks`.

8.4 RLS (already in `schema.sql`)

- `tests` : student sees rows where `cohort_id` matches theirs OR is null, AND status active.
- `submissions` : student reads/writes only their own.
- `answers` : readable only via a submission the student owns.
- `questions.answer_key_index` : never selected by the student client.

8.5 Cloudinary (already wired — keep as-is)

```
// base64 → Blob → FormData → unsigned upload → secure_url
fetch(`https://api.cloudinary.com/v1_1/${CLOUDINARY_CLOUD}/image/upload`, { metho
```

Store the returned `secure_url` in `fileData` / `photoData` / `summaryPhoto` instead of `base64`.

9. Security model

Concern	Current (client build)	Production requirement
Student passwords	PBKDF2-SHA256 + per-user salt; legacy plaintext auto-migrates on load	Same, hashes only; never store plaintext
Admin auth	Password hash constant; demo uses <code>demo2026</code>	Move to Supabase Auth role/JWT
Brute force	<code>ht_admin_lock : 5 fails</code> → escalating cooldown (30s·2 ⁽ⁿ⁻⁵⁾ , max 10m)	Keep; ideally also rate-limit server-side
Idle session	30-min admin auto-logout (mousemove/keydown/click/touch reset)	Keep
MCQ answer key	Present in client memory (demo)	Must be server-only via trigger + RLS
Score integrity	Client computes totals	Must be <code>sum_marks_trigger</code>
Transport	—	HTTPS via Vercel (automatic)

Honest limit: a browser-only build cannot hide answer keys or guarantee score integrity.

The Supabase trigger + RLS path (§8) is what closes that gap. For small known batches the client build is acceptable; for anything graded competitively, finish the Supabase wiring.

10. Build & deploy (summary)

Full step-by-step with screenshots-level detail is in `HamzaTeaches-DeveloperBrief.docx`. In short:

```
# local
npm install
npm run dev          # http://localhost:3000

# deploy
npm install -g vercel
vercel               # paste 4 env vars from .env, accept defaults
```

`.env` (never commit):

```
REACT_APP_SUPABASE_URL=
REACT_APP_SUPABASE_ANON_KEY=
REACT_APP_CLOUDINARY_CLOUD=
REACT_APP_CLOUDINARY_PRESET=hamza_submissions
```

Run `schema.sql` once in the Supabase SQL editor. Region: Asia-Pacific (Singapore).

11. Acceptance criteria (per feature)

Student login

- Valid username+password → dashboard; invalid → inline error.
- Lockout message shows remaining seconds after 5 fails (prod).

Take test

- Timer counts down; refresh resumes same remaining time.
- Reaching 0 OR passing `closeAt` auto-submits with `timeUp=true`.
- MCQ/text/photo all submit; double-submit blocked.

Results

- Hidden until `released=true`.
- Shows score, grade band, per-question correctness, feedback, MCQ correct answer.

Create test

- Validation per §7.9 blocks save with field errors.
- Saved as `draft` or `active`; unique 4-char code generated.

Grade

- MCQ marks auto-filled; text/photo accept 0..max.
- "Release" sets `released=true` and surfaces results to the student.
- Re-grading after release updates the score.

Bulk release

- Offered only when all questions are MCQ and unreleased submissions exist.

Cohorts

- Admin cohort filter scopes dashboard, analytics, and test visibility.
- Students only see tests for their cohort or `cohortId=null`.

Analytics

- Trend, topic mastery, and grade distribution recompute from released submissions only.
- Weak/critical topics flagged per §7.2.

Security (prod gate)

- No `correctOption / answer_key_index` in any student network response.
- Submission totals come from the trigger, not the client.

12. Constraints & non-goals

In scope: timed tests (MCQ/text/photo), manual + auto grading, per-question feedback, release control, cohorts, announcements, question bank, score/topic analytics, JSON export.

Not built / out of scope: student self-signup, AI grading, plagiarism detection, live proctoring, parent portal, messaging, offline mode, print/PDF export of results, third-party

API.

Hard rules: no em-dashes in any UI copy; never describe Hamza as “qualified accountant” or “ACCA member” (he is an **ACCA affiliate**); never embed real Cambridge past-paper content — all questions original.